E N S A · B E N I M E L L A L

Pr. BE ELBAGHAZAOUI

C O M P T E R E N D U

Travaux Pratiques — Architecture Logicielle

TP 1	Application Monolithique Simple — Spring Boot
TP 2	Monolithe Modulaire — DTO, MapStruct, Service Layer
TP 3	Architecture Distribuée — Nginx, MongoDB, Redis, RabbitMQ

Adam El Mansour

Année universitaire 2025–2026

TP1 — Application Monolithique Simple

1. Objectifs

- Créer une application Spring Boot monolithique simple
- Développer une API REST CRUD complète
- Comprendre l'architecture en couches : Entity, Repository, Service, Controller
- Découvrir RestRepository pour simplifier le code
- Identifier les avantages et limites d'un monolithe

2. Technologies utilisées

Technologie	Rôle
Spring Boot	Framework principal
Spring Data JPA	Accès base de données
PostgreSQL	Base de données relationnelle
Lombok	Réduction du code boilerplate
Postman	Test des endpoints REST

3. Architecture en couches

L'application suit l'architecture classique en 4 couches :

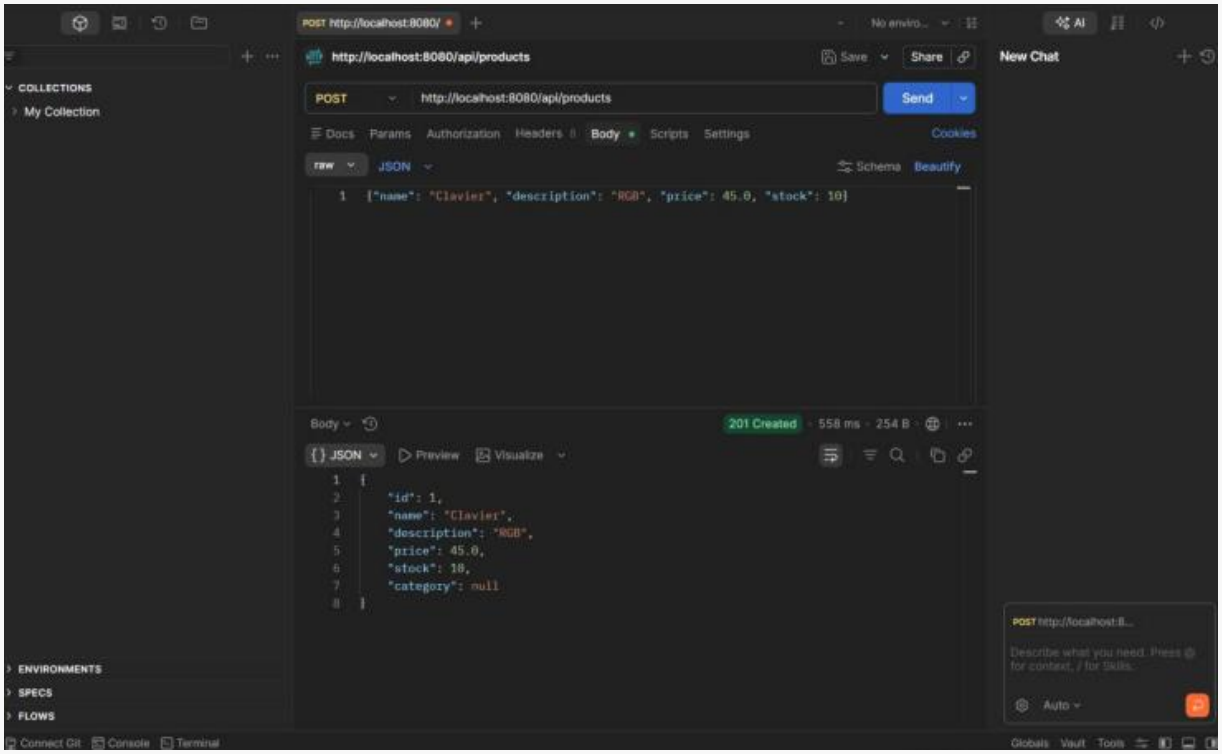
- Entity (Model) : représente la table products en base de données
- Repository : interface d'accès aux données via JpaRepository
- Service : logique métier (CRUD)
- Controller : exposition des endpoints REST

4. Endpoints REST

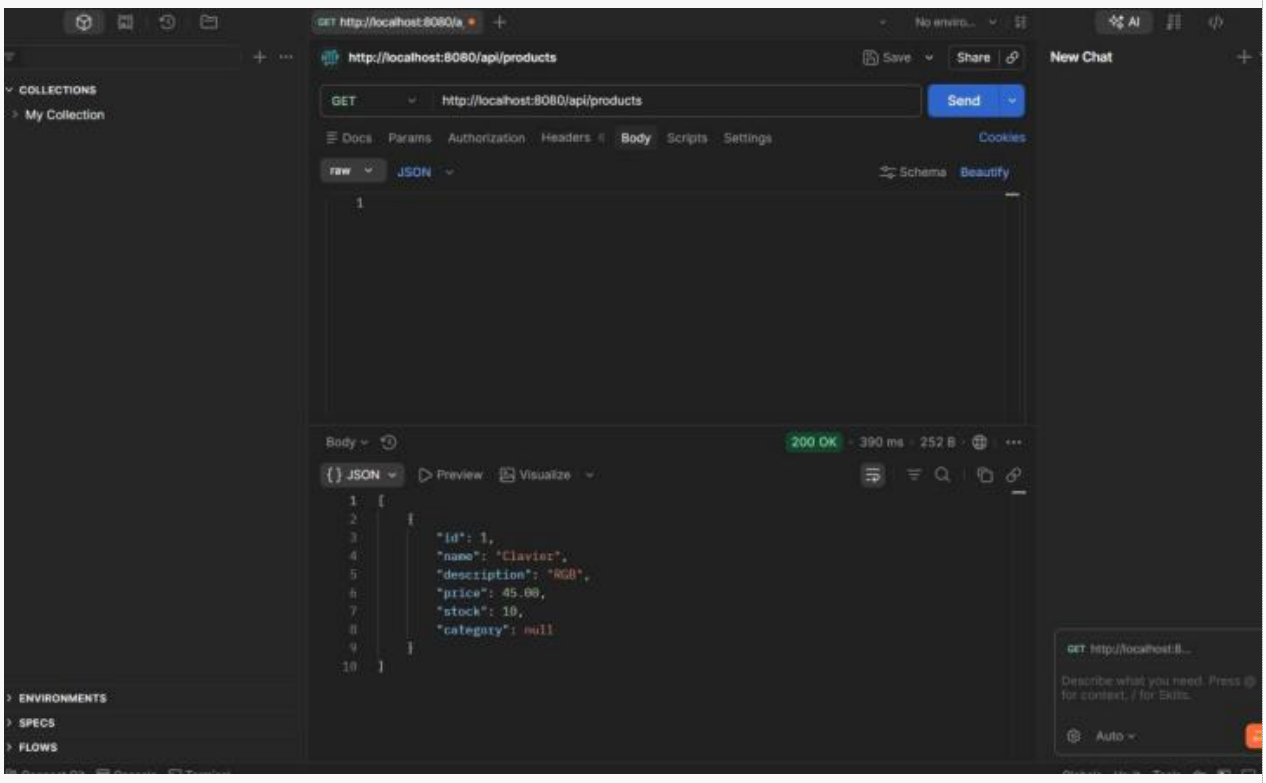
Méthode	URL	Description
GET	/api/products	Liste tous les produits
GET	/api/products/{id}	Récupère un produit par ID
POST	/api/products	Créer un nouveau produit
PUT	/api/products/{id}	Modifier un produit
DELETE	/api/products/{id}	Supprimer un produit

5. POST/GET/PUT/DELETE

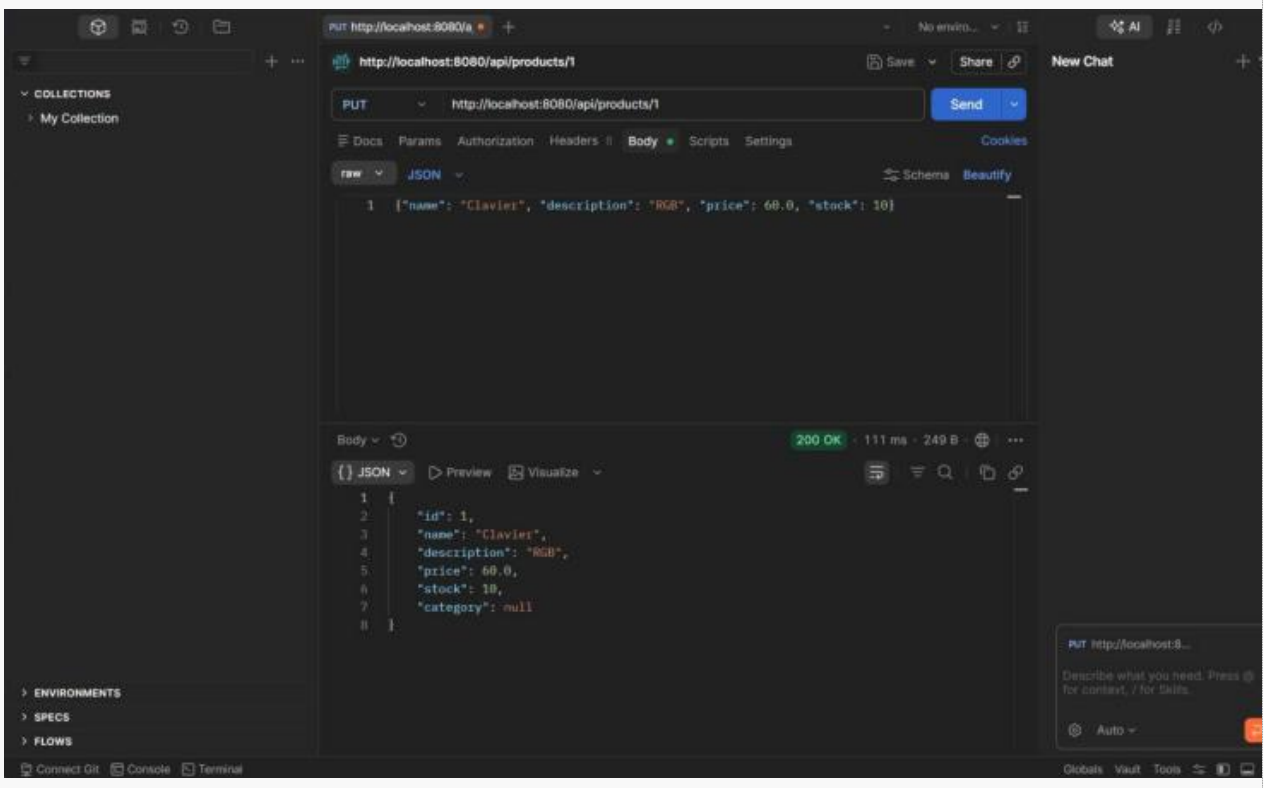
5.1 Création d'un produit (POST)



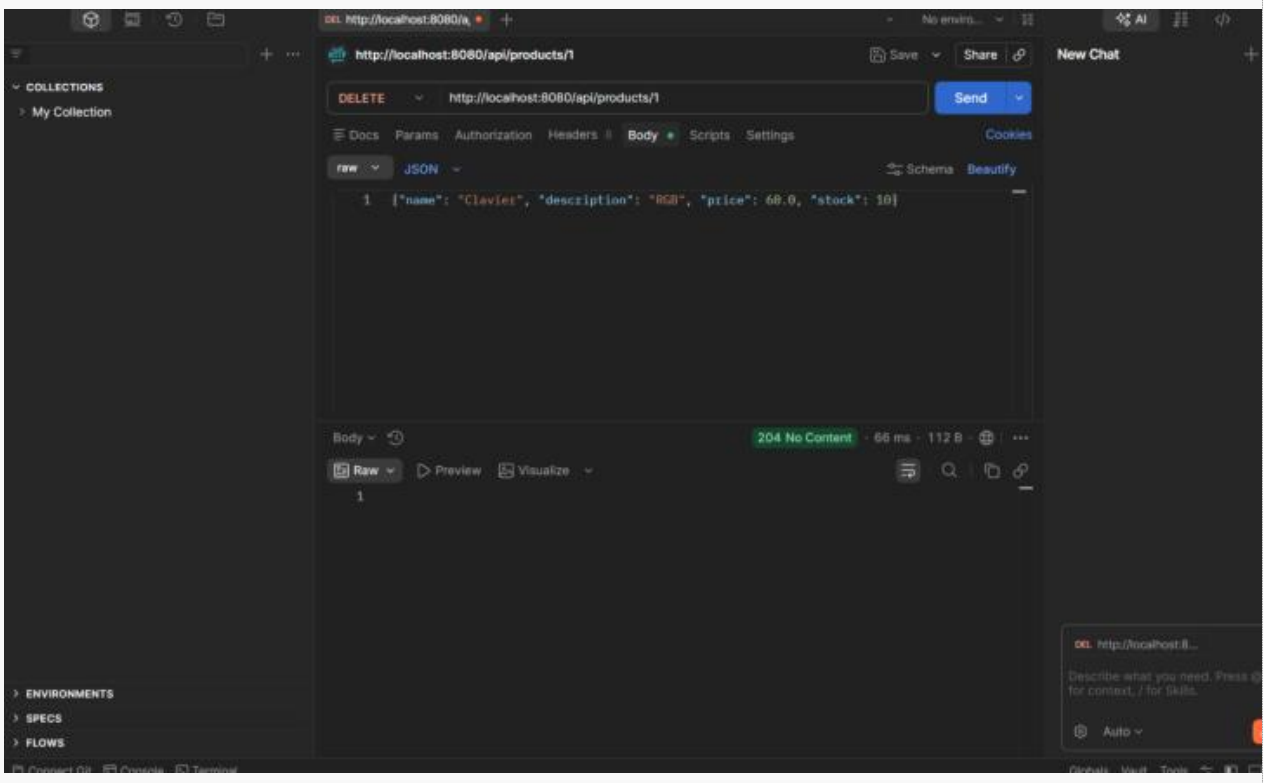
5.2 Liste des produits (GET)



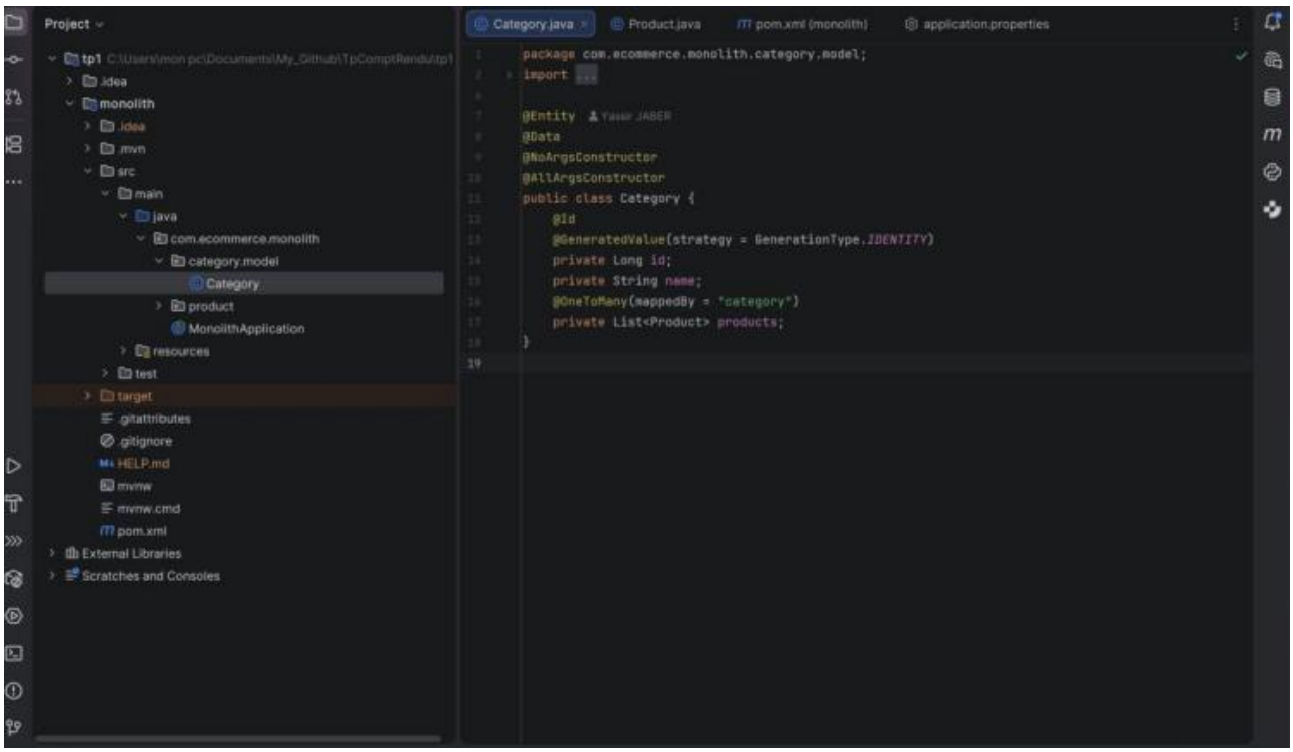
5.3 Modification d'un produit (PUT)



5.4 Suppression d'un produit (DELETE)



6. EXERCICE :


Project
tp1 C:\Users\mon pc\Documents\My_Github\TpComptRendu\tp1
.idea
monolith
.idea
.mvn
src
main
java
com.ecommerce.monolith
category.model
Category
product
MonolithApplication
resources
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
External Libraries
Scratches and Consoles
Category.java
Product.java
pom.xml (monolith)
application.properties
package com.ecommerce.monolith.category.model;
import ...
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Category {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String name;
@OneToMany(mappedBy = "category")
private List<Product> products;
}

TP2 — Monolithe Modulaire

1. Objectifs

- Restructurer le monolithe en modules métier distincts
- Définir des boundaries claires entre domaines
- Implémenter des interfaces de communication entre modules
- Appliquer les design patterns : DTO, Mapper, Service Layer
- Préparer le terrain pour une future migration microservices

2. Nouvelle structure modulaire

Module	Contenu
product	Controller, Service, Repository, Model, DTO, Mapper
order	Controller, Service, Repository, Model, DTO, Mapper
customer	Controller, Service, Repository, Model, DTO, Mapper

3. Patterns utilisés

3.1 DTO (Data Transfer Object)

Séparation entre l'entité JPA (Product) et l'objet exposé à l'API (ProductDTO). Cela protège la couche de données et permet d'exposer uniquement les champs nécessaires.

3.2 MapStruct

MapStruct génère automatiquement le code de mapping entre Entity et DTO à la compilation, sans réflexion Java à l'exécution — très performant.

3.3 Interface Service

Le service est défini comme interface (ProductService) et implémenté par ProductServiceImpl. Cela favorise le découplage et facilite les tests unitaires.

4. Le code + exercice

Voir github avec le lien ici :

https://github.com/adammansourxyz69420-max/tp-logicielle

TP3 — Architecture Distribuée

1. Objectifs

- Déployer une architecture distribuée complète avec 6 composants interconnectés
- Comprendre le rôle de chaque composant : Client, Load Balancer, Serveur, Cache, BDD, Bus de messages
- Implémenter un flux complet : POST commande → MongoDB → Redis → RabbitMQ
- Tester et observer le comportement distribué avec Postman

2. Architecture du système

Composant	Technologie	Rôle
Client	Postman	Envoie les requêtes HTTP POST/GET
Load Balancer	Nginx :80	Répartit le trafic (Round-robin)
Serveur applicatif	Node.js / Express	Logique métier, 2 instances
Base de données	MongoDB	Stockage persistant des commandes
Cache	Redis	Résultats GET en cache (TTL 30s)
Bus de messages	RabbitMQ	Notifications asynchrones

3. Flux de données

- Postman → Nginx :80 → Node.js :3001 ou :3002
- Node.js → MongoDB (stockage persistant)
- Node.js → Redis (invalidation du cache POST, lecture cache GET)
- Node.js → RabbitMQ → Consommateur → Notification console

4. Déploiement Docker

4.1 Build et démarrage des conteneurs

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

[+] up 56/56esolving provenance for metadata file
 ✓ Image rabbitmq:3-management           Pulled
 ✓ Image mongo:6                         Pulled
 ✓ Image redis:7-alpine                  Pulled
 ✓ Image nginx:alpine                    Pulled
 ✓ Image tp-archi-distribuee-app1        Built
 ✓ Image tp-archi-distribuee-app2        Built
 ✓ Network tp-archi-distribuee_default   Created
 ✓ Volume tp-archi-distribuee_mongo_data Created
 ✓ Container redis                       Created
 ✓ Container mongodb                     Created
 ✓ Container rabbitmq                    Created
 ✓ Container app2                        Created
 ... 2 more
```

Figure 1 — docker-compose up --build : toutes les images téléchargées et conteneurs créés

4.2 Vérification des 6 conteneurs actifs

```
PS C:\Users\msi\tp-archi-distribuee> docker ps -a
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS          PORTS
                                                                   NAMES
ac198810fca0   nginx:alpine              "/docker-entrypoint.…"   25 seconds ago   Up 23 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp
                                                                   nginx
db3fe735cb95   tp-archi-distribuee-app1  "docker-entrypoint.s…"   25 seconds ago   Up 23 seconds   0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
                                                                   app1
955c66b3ddb2   tp-archi-distribuee-app2  "docker-entrypoint.s…"   25 seconds ago   Up 23 seconds   0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
                                                                   app2
76c5f8f44acb   redis:7-alpine            "docker-entrypoint.s…"   25 seconds ago   Up 24 seconds   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
                                                                   redis
7e385502660e   rabbitmq:3-management     "docker-entrypoint.s…"   25 seconds ago   Up 24 seconds   0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp
, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp   rabbitmq
8e07461ac59c   mongo:6                   "docker-entrypoint.s…"   25 seconds ago   Up 24 seconds   0.0.0.0:27017->27017/tcp, [::]:27017->27017
/tcp                                                               mongodb
PS C:\Users\msi\tp-archi-distribuee>
```

Figure 2 — docker ps : les 6 conteneurs en statut UP (nginx, app1, app2, redis, rabbitmq, mongodb)

4.3 Health check

```
Impression élégante

{"status":"OK","instance":"Instance-1"}
```

Figure 3 — http://localhost/health — Réponse OK de Instance-1 via Nginx

5. Tests avec Postman

5.1 Création d'une commande (POST /commandes)

Méthode : POST | URL : http://localhost/commandes | Body : JSON

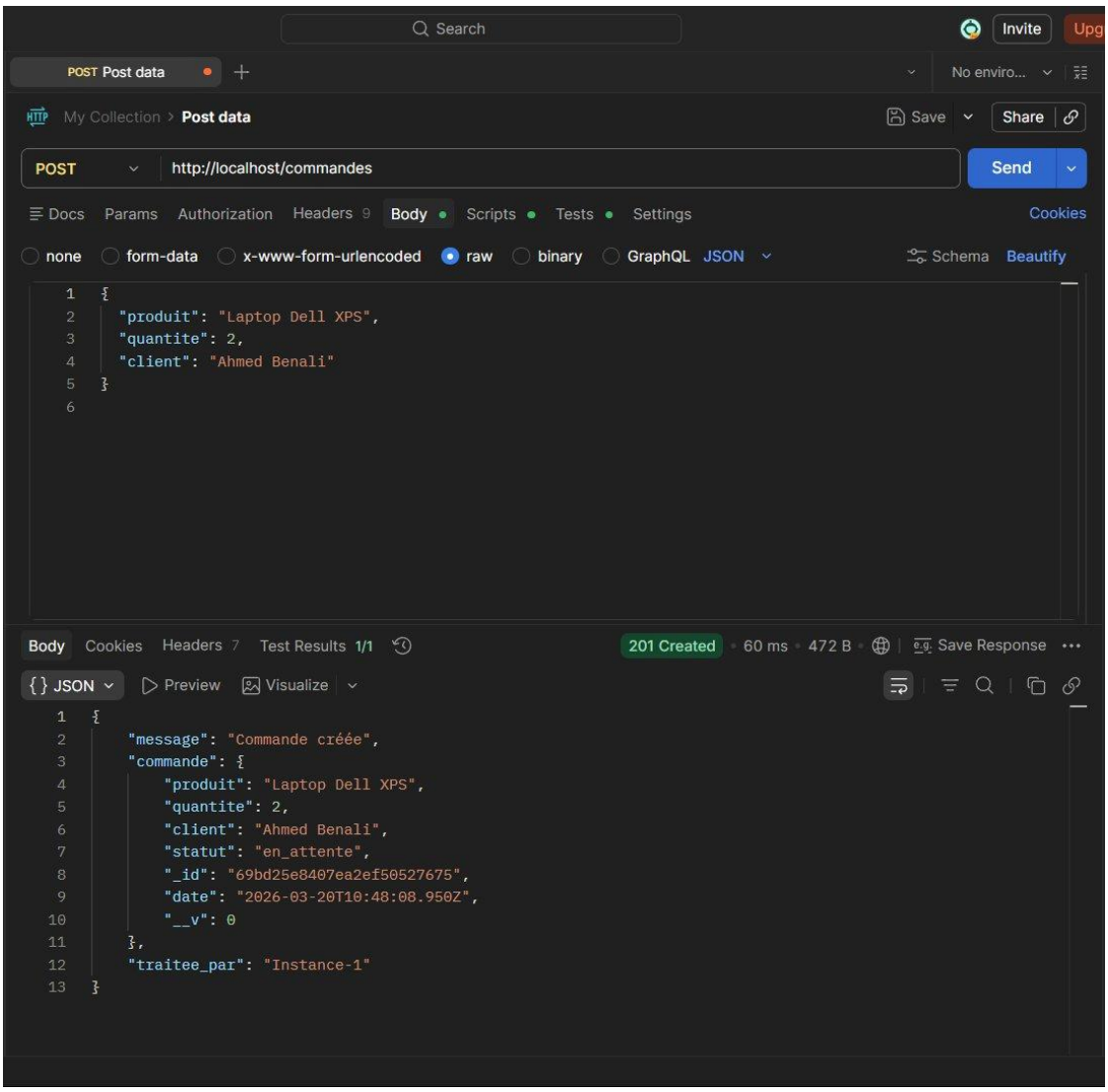


Figure 4 — POST /commandes — 201 Created — Commande sauvegardée dans MongoDB — traitee_par: Instance-1

5.2 Cache Redis — 1er appel (source MongoDB)

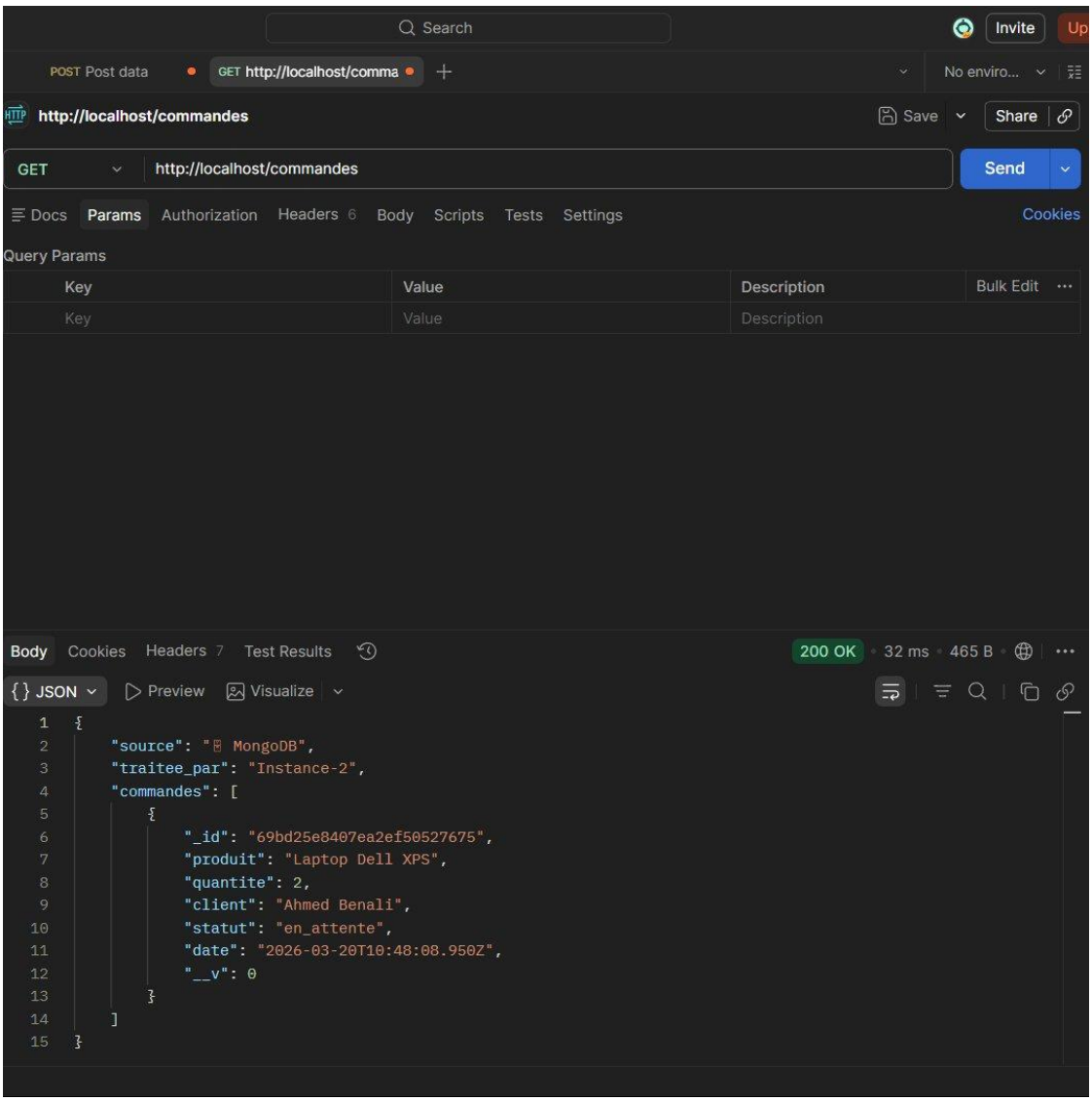


Figure 5 — GET /commandes — 1er appel : source MongoDB — données récupérées et mises en cache Redis (TTL 30s)

5.3 Cache Redis — 2ème appel (source Cache)

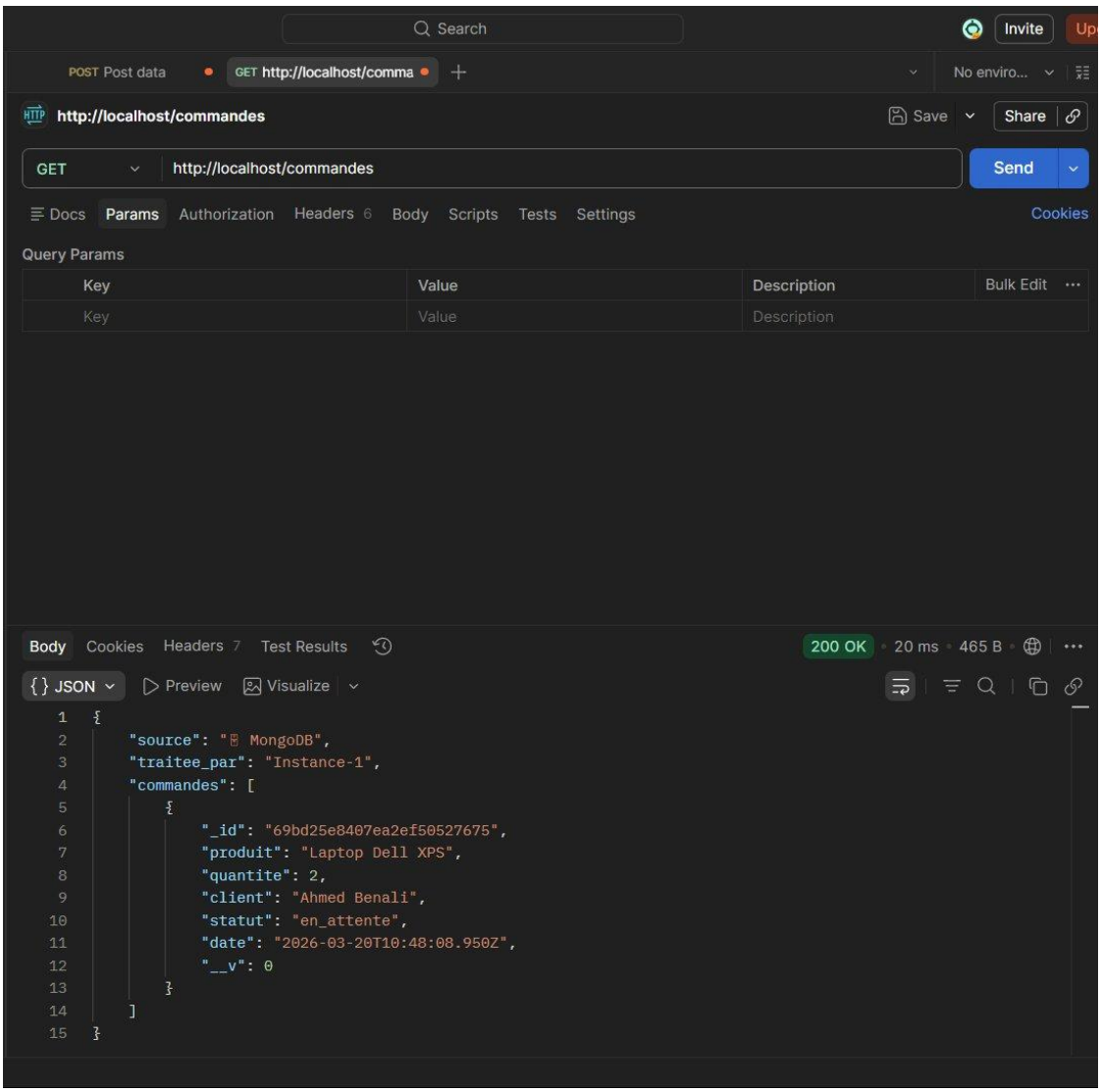


Figure 6 — GET /commandes — 2ème appel : source CACHE Redis — réponse plus rapide (~20ms vs ~32ms)

6. Load Balancing Round-Robin

4 requêtes POST successives envoyées pour observer l'alternance entre Instance-1 et Instance-2 :

Requête	Produit	Client	Instance
1	Laptop Dell XPS	Ahmed Benali	Instance-1 ✓
2	iPhone 15	Sara Alami	Instance-1 *
3	Clavier Logitech	Fatima Zahra	Instance-2 ✓
4	Souris Gaming	Youssef Bennani	Instance-1 ✓

* app2 était encore en train de se connecter à RabbitMQ au démarrage.

▸ 6.1 Requête 1 — Instance-1

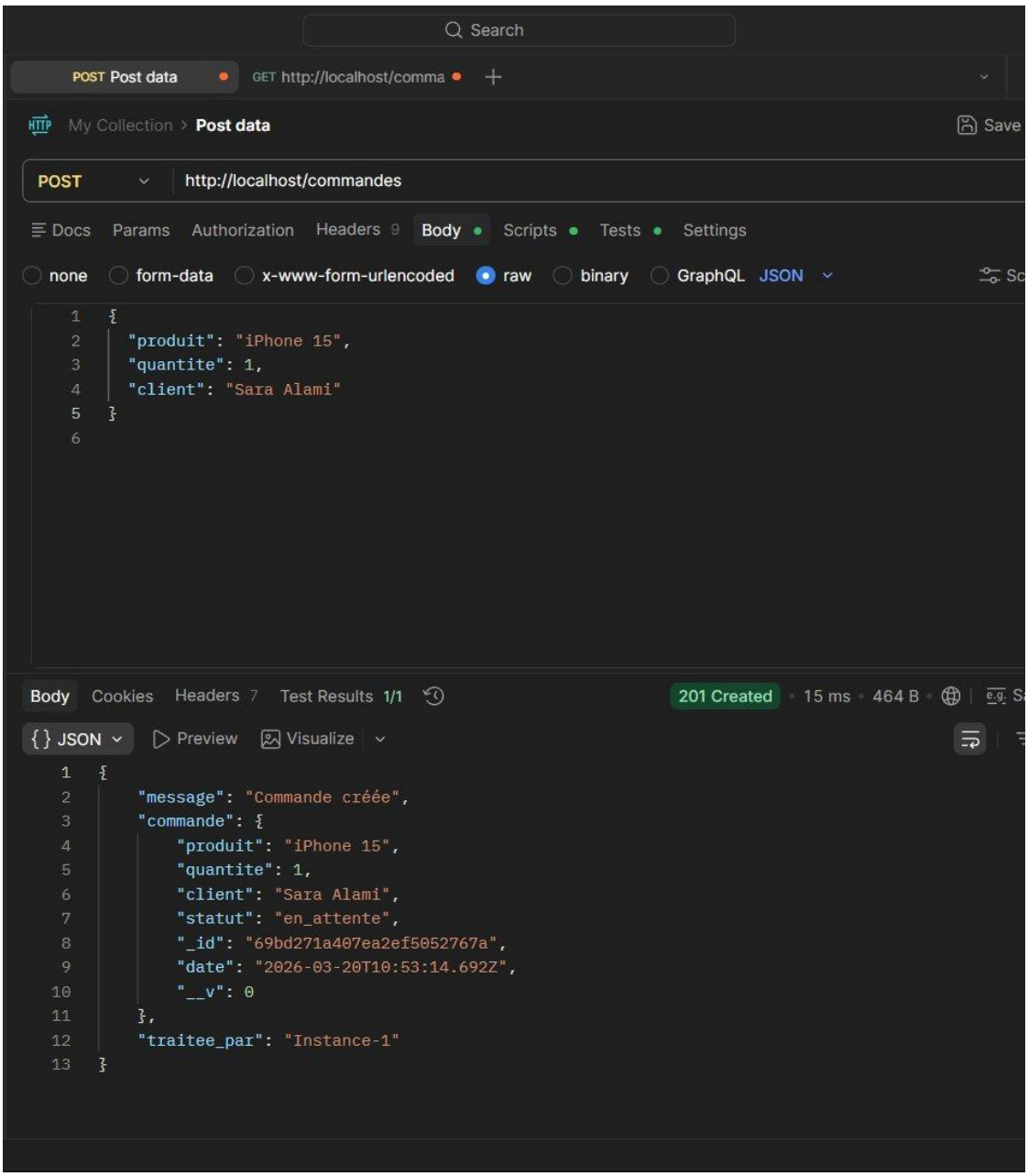


Figure 7 — Requête 1 (iPhone 15) → traitee_par: Instance-1

6.2 Requête 2 — Instance-1 (app2 en démarrage)

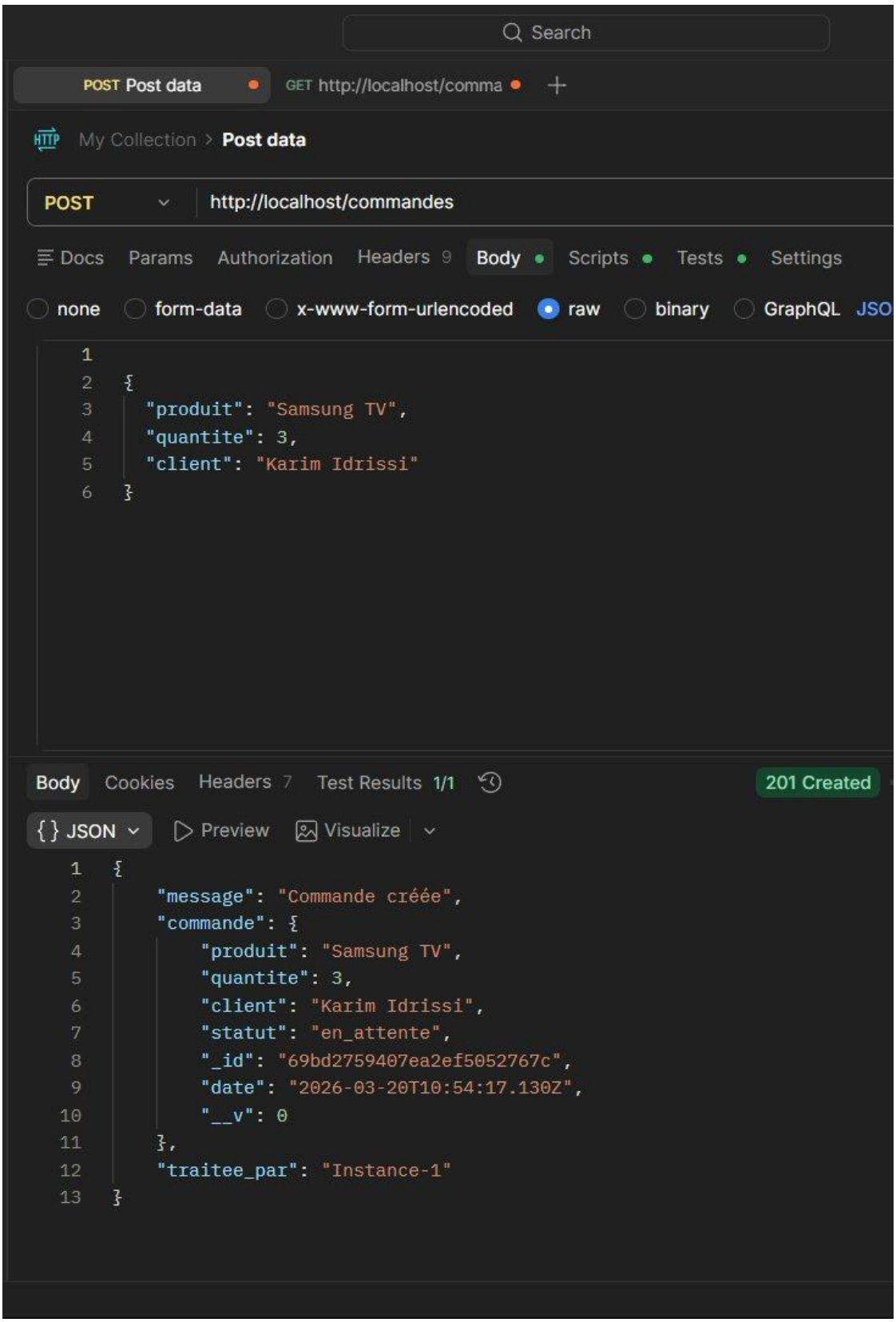


Figure 8 — Requête 2 (Samsung TV) → traitee_par: Instance-1

6.3 Requête 3 — Instance-2

Search

POST Post data GET http://localhost/comma

My Collection > **Post data**

POST http://localhost/commandes

Docs Params Authorization Headers 9 Body Scripts

none form-data x-www-form-urlencoded raw b

```
1  {
2    "produit": "Clavier Logitech",
3    "quantite": 5,
4    "client": "Fatima Zahra"
5  }
```

Body Cookies Headers 7 Test Results 1/1

{} JSON Preview Visualize

```
1  {
2      "message": "Commande créée",
3      "commande": {
4          "produit": "Clavier Logitech",
5          "quantite": 5,
6          "client": "Fatima Zahra",
7          "statut": "en_attente",
8          "_id": "69bd278ee018f83abe6f5587",
9          "date": "2026-03-20T10:55:10.523Z",
10         "__v": 0
11     },
12     "traitee_par": "Instance-2"
13 }
```

Figure 9 — Requête 3 (Clavier Logitech) → traitee_par: Instance-2 ✓Round-robin actif

6.4 Requête 4 — Instance-1

Search

POST Post data · GET http://localhost/comma

My Collection > Post data

POST http://localhost/commandes

Docs Params Authorization Headers 9 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "produit": "Souris Gaming",
3   "quantite": 2,
4   "client": "Youssef Bennani"
5 }
```

Body Cookies Headers 7 Test Results 1/1 201 Created 9 ms

{} JSON Preview Visualize

```
1  {
2      "message": "Commande créée",
3      "commande": {
4          "produit": "Souris Gaming",
5          "quantite": 2,
6          "client": "Youssef Bennani",
7          "statut": "en_attente",
8          "_id": "69bd27b5407ea2ef5052767e",
9          "date": "2026-03-20T10:55:49.961Z",
10         "__v": 0
11     },
12     "traitee_par": "Instance-1"
13 }
```

Figure 10 — Requête 4 (Souris Gaming) → traitee_par: Instance-1 ✓Alternance confirmée

7. Réponses aux questions du compte rendu

▸ Q1 — Quel composant reçoit en premier les requêtes ? Quel algorithme ?

C'est Nginx (le Load Balancer) qui reçoit en premier toutes les requêtes sur le port 80. Il utilise l'algorithme Round-robin par défaut, distribuant les requêtes en alternance : req1→app1, req2→app2, req3→app1…

▸ Q2 — Que se passe-t-il si app1 tombe en panne ?

Nginx détecte que app1 ne répond plus et redirige automatiquement toutes les requêtes vers app2. Le système continue de fonctionner sans interruption de service. C'est l'un des avantages clés de l'architecture distribuée : la résilience par redondance.

▸ Q3 — Différence entre 1er GET (MongoDB) et 2ème GET (Redis)

Le 1er GET interroge directement MongoDB (~32ms) et stocke le résultat dans Redis avec un TTL de 30 secondes. Le 2ème GET lit depuis Redis (~20ms) sans accéder à MongoDB — la réponse est plus rapide car les données sont déjà en mémoire.

▸ Q4 — Pourquoi invalider le cache Redis après un POST ?

Après un POST /commande, une nouvelle entrée est ajoutée en MongoDB. Si on conserve le cache, le prochain GET retournerait des données obsolètes sans la nouvelle commande. On supprime donc la clé 'toutes_commandes' pour forcer une nouvelle lecture depuis MongoDB.

▸ Q5 — En quoi RabbitMQ améliore-t-il la résilience ?

RabbitMQ découple la création de commande de l'envoi de notifications. Si le consommateur tombe, les messages s'accumulent dans la queue (durable:true) et sont traités au redémarrage. Cela évite la perte de données et les dépendances synchrones bloquantes entre services.

▸ Q6 — Ajouter une 3ème instance dans nginx.conf

Il suffit d'ajouter une ligne dans le bloc upstream backend de nginx.conf :

```
upstream backend {
  server app1:3001;
  server app2:3002;
  server app3:3003;   ← ajouter cette ligne
}
```